

数字电路设计第 5 章—硬件描述语言 HDL (DDCA Ch4)

考试导向学习笔记 | 与实验 3 单周期 CPU 紧密相关

中南大学计算机学院

2026 年 6 月 16 日

目录

1 考试重要性总览	3
2 引言——硬件描述语言 (Introduction)	4
3 SystemVerilog 模块基础 (Modules)	5
4 组合逻辑 HDL 描述 (Combinational Logic)	6
5 时序逻辑 HDL 描述 (Sequential Logic)	10
5.1 D 触发器 (D Flip-Flop)	10
5.2 带同步复位的 D 触发器	10
5.3 带异步复位的 D 触发器	11
5.4 带使能的 D 触发器	11
5.5 锁存器 (Latch) ——警告!	12
6 更多组合逻辑 HDL 写法 (More Combinational Logic)	13
7 阻塞赋值 vs 非阻塞赋值 (Blocking vs Nonblocking) 一必考!	15
8 有限状态机 FSM 的 HDL 实现 (Finite State Machines)	17
9 参数化模块 (Parameterized Modules)	19
10 测试平台 Testbench	20
10.1 简单 Testbench	20
10.2 自检 Testbench	20
10.3 基于测试向量的 Testbench	21
11 考点速查表	23

12 实验 3 关联：单周期 CPU 中的 HDL 应用

25

1 考试重要性总览

(与实验 3 CPU 相关)

本章是实验 3 (单周期 CPU 仿真) 的基础, 老师已做完整演示。考试重点关注以下核心考点:

考点	重要程度	与实验 3 关联
阻塞赋值 vs 非阻塞赋值	必考	时序逻辑正确性
FSM 三模块结构	高频	控制器设计
always_ff / always_comb 语法	必考	CPU 组合 + 时序逻辑
Module 结构建模 (实例化)		CPU 层次化设计
Testbench 编写		CPU 仿真验证
parameter 参数化模块		位宽复用

考试提示: 书上特别强调第 4.6 节 "Blocking vs. Nonblocking Assignment" (p.48) 是理解 HDL 时序行为的关键。如果弄错 `=` 和 `<=`, 综合出的硬件完全不同。

2 引言——硬件描述语言 (Introduction)

[p.3–9]

什么是 HDL?

[p.3] HDL (Hardware Description Language) 是一种描述数字电路功能的编程语言，由 CAD 工具自动综合为门级网表。

- HDL 只描述 **逻辑功能** (logic function)，不管具体晶体管如何实现
- CAD 工具 (如 Vivado、Design Compiler) 将 HDL 代码**综合** (synthesis) 成优化的门级电路
- 现代商业数字芯片几乎 100% 使用 HDL 设计

SystemVerilog vs VHDL

[p.3]

	SystemVerilog / Verilog	VHDL
开发者	Gateway Design Automation (1984)	美国国防部 (1981)
IEEE 标准	IEEE STD 1364-1995, 1800-2009	IEEE STD 1076-1987, 2008
特点	语法类似 C，简洁灵活	语法类似 Ada，类型严格
本书使用	SystemVerilog	—

本学期采用 SystemVerilog，与 Digital 工具和 iVerilog 仿真器配套。

仿真 vs 综合 (Simulation vs Synthesis)

[p.4–5]

- **仿真 (Simulation)**: 给电路施加激励输入，检查输出是否正确。在仿真中调试 bug，节省数百万美元——避免流片后才在硬件上发现问题。
- **综合 (Synthesis)**: 将 HDL 代码转换为**网表** (netlist) ——门级元件及其连接关系的列表。

实验环境: Digital (画图 + 导出 Verilog) + iVerilog (仿真) + GTKWave (看波形) + VSCode (编辑代码)。Digital 和 GTKWave 是两个独立工具，不能互相配置。

关键提醒 (Important!)

[p.5]

使用 HDL 时，脑中必须时刻想着 HDL 应该综合出什么样的硬件！

这就是所谓的“Digital HDL”思维——设计思路和 HDL 代码结合。

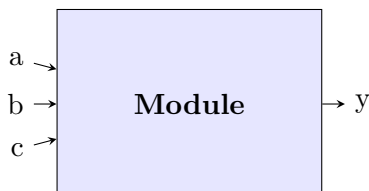
3 SystemVerilog 模块基础 (Modules)

[p.10–16]

模块 (Module) 概念

[p.10]

模块是 SystemVerilog 的基本设计单元，类似于 C 语言中的函数，但描述的是硬件块。



两种模块类型：

1. 行为建模 (Behavioral)：描述模块做什么（用 assign、always 等描述逻辑功能）
2. 结构建模 (Structural)：描述模块如何由更简单的子模块构成（层次化实例化）

基本语法规则

[p.13, 16]

- 大小写敏感：reset 和 Reset 是不同的信号
- 不允许以数字开头：2mux 是非法标识符
- 空白符忽略：缩进和空格不影响综合结果
- 注释：单行用 //，多行用 /* ... */
- 模块边界：module / endmodule

第一个模块示例

[p.11, 13]

```

1 module example(input logic a, b, c,
2                 output logic y);
3   assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;
4 endmodule
  
```

- module / endmodule 包围模块定义
- example 是模块名称
- 端口方向：input / output
- 端口类型：logic（SystemVerilog 新增的数据类型）
- assign 语句：连续赋值，用于组合逻辑（持续赋值 continuous assignment）
- 运算符：~ (NOT), & (AND), | (OR)

4 组合逻辑 HDL 描述 (Combinational Logic)

[p.17–35]

结构建模——层次化 (Structural Modeling)

[p.17]

将基本模块实例化以构建更大系统：

```

1 module and3(input logic a, b, c,
2             output logic y);
3     assign y = a & b & c;          // 行为建模
4 endmodule
5
6 module inv(input logic a,
7            output logic y);
8     assign y = ~a;                // 行为建模
9 endmodule
10
11 module nand3(input logic a, b, c,
12              output logic y);
13     logic n1;                     // 内部信号
14     and3 andgate(a, b, c, n1);    // 结构建模：实例化and3
15     inv inverter(n1, y);          // 结构建模：实例化inv
16 endmodule

```

关键点：底层模块用行为建模（描述功能），上层模块用结构建模（实例化连接）。

按位运算符 (Bitwise Operators)

[p.18]

处理多 bit 宽度的总线信号：

```

1 module gates(input logic [3:0] a, b,
2              output logic [3:0] y1, y2, y3, y4, y5);
3     assign y1 = a & b;    // AND
4     assign y2 = a | b;    // OR
5     assign y3 = a ^ b;    // XOR
6     assign y4 = ~(a & b); // NAND
7     assign y5 = ~(a | b); // NOR
8 endmodule

```

位宽声明：[3:0] 表示 4-bit 总线，按位操作应用于每一位。

缩减运算符 (Reduction Operators)

[p.19]

将多 bit 信号缩减为单 bit 结果:

```

1 module and8(input logic [7:0] a,
2             output logic y);
3   assign y = &a; // 等同于 a[7] & a[6] & ... & a[0]
4 endmodule

```

所有缩减运算符: & (AND), |& (NAND), | (OR), | (NOR), ^ (XOR), ~^ (XNOR)。

条件赋值 (Conditional Assignment / 三元运算符)

[p.20]

```

1 module mux2(input logic [3:0] d0, d1,
2             input logic s,
3             output logic [3:0] y);
4   assign y = s ? d1 : d0; // s=1选d1, s=0选d0
5 endmodule

```

?: 是三元运算符, 三个操作数为 s, d1, d0。

运算符优先级表 (Precedence)

[p.22]

优先级	运算符	含义
最高	~	NOT
	*, /, %	乘、除、取模
	+, -	加、减
	<<, >>	逻辑移位
	<<<, >>>	算术移位
	<, <=, >, >=	比较
	==, !=	相等、不等
	& / &	AND / NAND
	XOR / XNOR: ^, ~^	
	/ ↑	OR / NOR
	?:	三元条件
最低		

数字常量格式

[p.23]

格式: N'Bvalue, 其中 N= 位数, B= 进制

写法	位数	进制	十进制值	存储值
3'b101	3	二	5	101
'b11	不定	二	3	00...0011
8'b11	8	二	3	00000011
8'b1010_1011	8	二	171	10101011
3'd6	3	十	6	110
6'o42	6	八	34	100010
8'hAB	8	十六	171	10101011
42	不定	十	42	00...0101010

进制字母：b = 二进制, o = 八进制, d = 十进制, h = 十六进制。下划线 _ 仅用于可读性，被忽略。

位操作 (Bit Manipulations)

[p.24–25]

拼接运算符 {} 和重复运算符 {{n}}:

```
1 assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};
2 // 若y为12-bit, 产生: a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0
```

三态输出 / 高阻 Z (Floating Output)

[p.26]

```
1 module tristate(input logic [3:0] a,
2               input logic en,
3               output tri [3:0] y);
4   assign y = en ? a : 4'bz; // en=0时输出高阻态
5 endmodule
```

三态 (tri-state): 输出可为 0, 1, 或 Z (高阻/浮空), 用于总线共享。

延迟 (Delays)

[p.27, 31–35]

```
1 module example(input logic a, b, c,
2               output logic y);
3   logic ab, bb, cb, n1, n2, n3;
4   assign #1 {ab, bb, cb} = ~{a, b, c}; // 1单位延迟
5   assign #2 n1 = ab & bb & cb;        // 2单位延迟
6   assign #2 n2 = a & bb & cb;
7   assign #2 n3 = a & bb & c;
8   assign #4 y = n1 | n2 | n3;        // 4单位延迟
9 endmodule
```


延迟仅用于仿真（模拟门传播延迟），综合时被忽略。

5 时序逻辑 HDL 描述 (Sequential Logic)

[p.36–42]

HDL 习语 (Idioms)

[p.36]

关键提醒： SystemVerilog 使用特定习语 (idioms) 来描述锁存器、触发器和 FSM。其他编码风格可能仿真正确，但综合出错误的硬件！

always 语句结构

[p.37]

```
1 always @(sensitivity list)
2     statement;
```

每当敏感列表中的信号变化，语句就被执行。对应的 always 变体：

- always_ff —— 描述触发器（边沿敏感）
- always_comb —— 描述组合逻辑（电平敏感，自动推导敏感列表）
- always_latch —— 描述锁存器

5.1 D 触发器 (D Flip-Flop)

[p.38]

```
1 module flop(input logic clk,
2             input logic [3:0] d,
3             output logic [3:0] q);
4     always_ff @(posedge clk)
5         q <= d;           // 非阻塞赋值，q "接收" d
6 endmodule
```

关键点：

- always_ff @(posedge clk) 表示 clk 上升沿触发
- q <= d 是非阻塞赋值，读作“q gets d”

5.2 带同步复位的 D 触发器

[p.39]

```
1 module flopr(input logic clk,
2             input logic reset,
3             input logic [3:0] d,
4             output logic [3:0] q);
5     // 同步复位 (synchronous reset)
```

```

6  always_ff @(posedge clk)
7      if (reset) q <= 4'b0;
8      else      q <= d;
9  endmodule

```

同步复位：复位仅在时钟上升沿检查，敏感列表中只有 `clk`。

5.3 带异步复位的 D 触发器

[p.40]

```

1  module flopr(input logic clk,
2              input logic reset,
3              input logic [3:0] d,
4              output logic [3:0] q);
5  // 异步复位 (asynchronous reset)
6  always_ff @(posedge clk, posedge reset)
7      if (reset) q <= 4'b0;
8      else      q <= d;
9  endmodule

```

异步复位 vs 同步复位的区别：

特性	同步复位	异步复位
敏感列表	@(posedge clk)	@(posedge clk, posedge reset)
复位响应	仅在时钟上升沿有效	复位信号上升沿立即生效
FPGA 常用	推荐	也可用，需注意时序

5.4 带使能的 D 触发器

[p.41]

```

1  module flopren(input logic clk,
2                input logic reset,
3                input logic en,
4                input logic [3:0] d,
5                output logic [3:0] q);
6  // 使能 + 异步复位
7  always_ff @(posedge clk, posedge reset)
8      if (reset) q <= 4'b0;
9      else if (en) q <= d; // en=1才更新
10 endmodule

```

使能 (enable)：仅在 `en=1` 时数据更新；否则保持上一次的值。

5.5 锁存器 (Latch) ——警告!

[p.42]

```
1 module latch(input logic clk,  
2             input logic [3:0] d,  
3             output logic [3:0] q);  
4     always_latch  
5         if (clk) q <= d;    // 电平敏感, clk=1时透明  
6 endmodule
```

**警告：本书不使用锁存器！但你可能不经意写出隐含锁存器的代码。
如果综合出的硬件中有锁存器，说明代码有错误！**

锁存器意外产生的常见原因：

- `always_comb` 中未给所有分支赋值 (if 缺少 else)
- `case` 语句缺少 `default` 分支
- 组合逻辑中信号“回读”自己以前的值

6 更多组合逻辑 HDL 写法 (More Combinational Logic)

[p.43–47]

`always_comb` 用于组合逻辑

[p.44]

```

1 module gates(input logic [3:0] a, b,
2               output logic [3:0] y1, y2, y3, y4, y5);
3     always_comb
4     begin
5         y1 = a & b;    // 注意: 这里用 = (阻塞赋值)
6         y2 = a | b;
7         y3 = a ^ b;
8         y4 = ~(a & b);
9         y5 = ~(a | b);
10    end
11 endmodule

```

要点:

- `always_comb` 内部用 `=` (阻塞赋值)
- 需要 `begin/end` 包围多条语句
- 当代码简单时, 优先用 `assign` 更简洁

`case` 语句实现组合逻辑

[p.45–46]

七段数码管译码器:

```

1 module sevenseg(input logic [3:0] data,
2                 output logic [6:0] segments);
3     always_comb
4     case (data)
5         //      abc_defg
6         0: segments = 7'b111_1110;
7         1: segments = 7'b011_0000;
8         2: segments = 7'b110_1101;
9         3: segments = 7'b111_1001;
10        4: segments = 7'b011_0011;
11        5: segments = 7'b101_1011;
12        6: segments = 7'b101_1111;
13        7: segments = 7'b111_0000;
14        8: segments = 7'b111_1111;
15        9: segments = 7'b111_0011;
16        default: segments = 7'b000_0000; // 必须有default!

```

```
17     endcase
18 endmodule
```

case 语句规则:

- case 描述组合逻辑必须覆盖所有可能输入组合
- 务必使用 default 语句——否则可能综合出锁存器!

casez 语句 (带无关项)

[p.47]

```
1 module priority_casez(input logic [3:0] a,
2                       output logic [3:0] y);
3     always_comb
4     casez(a)
5         4'b1???: y = 4'b1000; // ? = 无关项 (don't care)
6         4'b01?: y = 4'b0100;
7         4'b001?: y = 4'b0010;
8         4'b0001: y = 4'b0001;
9         default: y = 4'b0000;
10    endcase
11 endmodule
```

casez 中的? 表示无关位, 在匹配时被忽略。优先匹配靠前的条目 (实现优先级编码)。

7 阻塞赋值 vs 非阻塞赋值 (Blocking vs Nonblocking) —必考!

[p.48–49]

这是本章最重要的考点，直接决定时序逻辑的正确性。

核心定义

[p.48]

特性	<code><=</code> (非阻塞赋值, Nonblocking)	<code>=</code> (阻塞赋值, Blocking)
执行方式	所有非阻塞赋值同时发生	按代码顺序依次执行
行为	并行 (parallel)	串行 (sequential)
用在哪里	<code>always_ff</code> 中	<code>always_comb</code> / <code>assign</code> 中
语法	<code>q <= d;</code> (q gets d)	<code>y = a & b;</code>

正确 vs 错误：同步器例子

[p.48]

```

1 // 正确! 使用非阻塞赋值      // 错误! 使用阻塞赋值
2 module syncgood(input logic clk, module syncbad(input logic clk,
3         input logic d,         input logic d,
4         output logic q);       output logic q);
5     logic n1;                 logic n1;
6     always_ff @(posedge clk)   always_ff @(posedge clk)
7     begin                     begin
8         n1 <= d; // 非阻塞      n1 = d; // 阻塞
9         q <= n1; // 非阻塞      q = n1; // 阻塞
10    end                       end
11 endmodule                   endmodule

```

分析:

- **syncgood (非阻塞 `<=`)**: `n1` 被赋值为上一个 `clk` 上升沿时的 `d` 值, `q` 被赋值为上一个 `clk` 上升沿时的 `n1` 值。效果: 正确的两级移位寄存器 (`d → n1 → q`)。
- **syncbad (阻塞 `=`)**: 在同一个 `clk` 边沿, 按顺序执行。`n1 = d` 先执行, `q = n1` 后执行。效果: `n1` 和 `q` 得到相同的 `d` 值, 退化为单级寄存器。没有延迟效果!

信号赋值规则总结 (考试重点!)

[p.49]

1. 同步时序逻辑: 使用 `always_ff @(posedge clk)` + 非阻塞赋值 `<=`

```

1     always_ff @(posedge clk)
2     q <= d; // 非阻塞

```

2. 简单组合逻辑：使用 连续赋值 `assign` + 阻塞赋值 `=`

```
1 assign y = a & b;
```

3. 复杂组合逻辑：使用 `always_comb` + 阻塞赋值 `=`

```
1 always_comb  
2 y = a & b; // 阻塞
```

4. 一个信号只能在一条 `always` 语句或一条 `assign` 语句中被赋值。（不允许多驱动）

考场口诀：时序用 `<=` 非阻塞，组合用 `=` 阻塞。

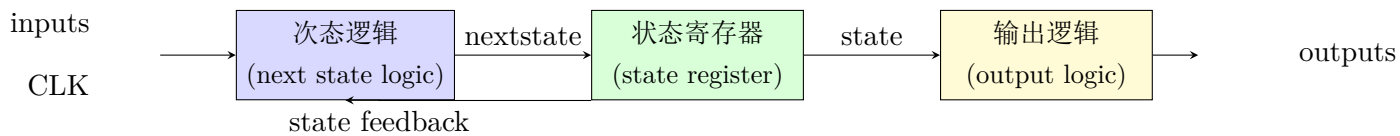
`always_ff` $\rightarrow$ `<=` | `always_comb` $\rightarrow$ `=` | `assign` $\rightarrow$ `=`

8 有限状态机 FSM 的 HDL 实现 (Finite State Machines)

[p.50–52]

FSM 三模块结构 (必考!)

[p.50]



三模块功能：

1. 次态逻辑 (next state logic): 根据当前状态和输入, 计算下一个状态 (组合逻辑)
2. 状态寄存器 (state register): 在时钟边沿更新状态 (时序逻辑)
3. 输出逻辑 (output logic): 根据当前状态产生输出 (组合逻辑)

FSM 示例: 除以 3 器 (Divide by 3)

[p.51–52]

状态转移图: $S0 \rightarrow S1 \rightarrow S2 \rightarrow S0 \rightarrow \dots$ (每 3 个时钟循环一次)

```

1 module divideby3FSM (input logic clk,
2                       input logic reset,
3                       output logic q);
4   typedef enum logic [1:0] {S0, S1, S2} statetype;
5   statetype [1:0] state, nextstate;
6
7   // 模块1: 状态寄存器 (时序逻辑, <= 非阻塞)
8   always_ff @(posedge clk, posedge reset)
9       if (reset) state <= S0;
10      else      state <= nextstate;
11
12   // 模块2: 次态逻辑 (组合逻辑, = 阻塞)
13   always_comb
14       case (state)
15           S0: nextstate = S1;
16           S1: nextstate = S2;
17           S2: nextstate = S0;
18           default: nextstate = S0;
19       endcase
20
21   // 模块3: 输出逻辑 (组合逻辑, assign)
22   assign q = (state == S0);
23 endmodule
  
```

FSM 实现要点总结:

- 使用 `typedef enum` 定义状态编码
- 状态寄存器用 `always_ff + <=` (非阻塞)
- 次态逻辑用 `always_comb + =` (阻塞)
- 输出逻辑可用 `assign` 或 `always_comb`
- 复位进入 S0 (双圈表示复位状态)

9 参数化模块 (Parameterized Modules)

[p.53]

通过参数化实现位宽可配置，提高代码复用性。

```
1 module mux2
2   #(parameter width = 8)           // 参数名和默认值
3   (input logic [width-1:0] d0, d1,
4    input logic s,
5    output logic [width-1:0] y);
6   assign y = s ? d1 : d0;
7 endmodule
```

实例化方式：

- 使用默认宽度 (8-bit): `mux2 myMux(d0, d1, s, out);`
- 指定宽度 (12-bit): `mux2 #(12) lowmux(d0, d1, s, out);`

在实验 3 中的应用：CPU 中多处需要参数化位宽（ALU、寄存器文件、多路选择器），通用参数化模块可大幅减少重复代码。

10 测试平台 Testbench

[p.54–66]

Testbench 概述

[p.54]

Testbench 是不可综合的 HDL 代码，用于验证待测模块（DUT/UUT）的功能。

三种类型：

1. 简单 Testbench：人工施加输入，手动观察波形
2. 自检 Testbench (Self-checking)：自动比较输出与期望值，打印错误
3. 基于测试向量的 Testbench (with Testvectors)：从文件读取测试向量，自动化测试

10.1 简单 Testbench

[p.55–57]

```

1 module testbench1();
2     logic a, b, c;
3     logic y;
4     // 实例化待测设备
5     sillyfunction dut(a, b, c, y);
6     // 逐一施加输入
7     initial begin
8         a = 0; b = 0; c = 0; #10;
9         c = 1;           #10;
10        b = 1; c = 0;     #10;
11        c = 1;           #10;
12        a = 1; b = 0; c = 0; #10;
13        c = 1;           #10;
14        b = 1; c = 0;     #10;
15        c = 1;           #10;
16    end
17 endmodule

```

特点：需要人工观察波形确认结果，不自动报告错误。

10.2 自检 Testbench

[p.58]

```

1 module testbench2();
2     logic a, b, c;
3     logic y;
4     sillyfunction dut(a, b, c, y); // 实例化dut
5     initial begin                // 施加输入，逐一检查结果

```

```

6   a = 0; b = 0; c = 0; #10;
7   if (y !== 1) $display("000_failed.");
8   c = 1;          #10;
9   if (y !== 0) $display("001_failed.");
10  // ... 其余省略 ...
11  end
12 endmodule

```

特点：使用 \$display 自动报告错误。比较器用 !== / ===（可比较 x 和 z 值）。

10.3 基于测试向量的 Testbench

[p.59–66]

四步流程：

1. 生成时钟：

```

1  always
2  begin
3      clk = 1; #5; clk = 0; #5; // 周期=10单位
4  end

```

2. 读取测试向量文件：

```

1  initial
2  begin
3      $readmemb("example.tv", testvectors); // 读取二进制向量
4      vectornum = 0; errors = 0;
5      reset = 1; #27; reset = 0;
6  end

```

3. 在时钟上升沿施加输入：

```

1  always @(posedge clk)
2  begin
3      #1; {a, b, c, yexpected} = testvectors[vectornum];
4  end

```

4. 在时钟下降沿比较输出：

```

1  always @(negedge clk)
2  if (~reset) begin
3      if (y !== yexpected) begin
4          $display("Error: _inputs_=%b", {a, b, c});
5          $display("_outputs_=%b_(_b_expected)", y, yexpected);
6          errors = errors + 1;
7      end
8      vectornum = vectornum + 1;
9      if (testvectors[vectornum] === 4'bx) begin
10         $display("%d_tests_completed_with_%d_errors",

```

```

11         vectornum, errors);
12     $finish;
13 end
14 end

```

Testbench 关键命令总结:

命令	功能
\$readmemb("file.tv", arr)	读取二进制测试向量文件到数组
\$readmemh("file.tv", arr)	读取十六进制测试向量文件
\$display("fmt", vars)	打印调试信息
\$finish	结束仿真
\$dumpfile("name.vcd")	指定波形转储文件
\$dumpvars(0, module)	转储所有信号到波形文件
#N	时间延迟 N 个单位

时序安排: 上升沿 vs 下降沿

[p.60, 64–65]

- 上升沿 @(posedge clk): 施加输入 (给 DUT 新激励)
- 下降沿 @(negedge clk): 比较输出 (等输出稳定后再检查)
- 原因: 上升沿输入变化, 组合逻辑有传播延迟, 下降沿时输出已稳定

11 考点速查表

基础概念

- **HDL 本质**: 描述逻辑功能 → CAD 综合为门级网表 [p.3]
- **仿真 vs 综合**: 仿真 = 验证功能 (省 \$); 综合 = 生成网表 [p.4-5]
- **模块类型**: Behavioral (描述功能) vs Structural (实例化连接) [p.10]
- **Module 语法**: module ... endmodule, 端口 input/output logic [p.11-16]

组合逻辑

- **bit 位宽总线**: [3:0] a = 4-bit, [7:0] = 8-bit [p.18]
- **按位运算符**: & | ^ 逐位操作 [p.18]
- **缩减运算符**: &a 将 a 所有 bit AND 为一个 bit [p.19]
- **三元条件**: s ? d1 : d0 二选一 MUX [p.20]
- **优先级表**: ~ > * / % > +- > 移位 > 比较 > & ^ | > ?: [p.22]
- **数字常量**: N'Bvalue, b= 二 / o= 八 / d= 十 / h= 十六 [p.23]
- **拼接 {}**: {a[2:1], {3{b[0]}}} 拼接 + 重复 [p.24]
- **Z 高阻**: 4'bz 三态输出, 总线 [p.26]
- **延迟 #N**: #1, #2, #4 仅仿真用, 不可综合 [p.27-35]

时序逻辑

- **always_ff**: 描述触发器, 边沿敏感, 用 <= [p.38]
- **always_comb**: 描述组合逻辑, 自动推导敏感列表, 用 = [p.44]
- **D 触发器**: always_ff @(posedge clk) q <= d; [p.38]
- **同步复位**: 敏感列表仅 clk, reset 在 if 中检查 [p.39]
- **异步复位**: 敏感列表含 clk+reset, 立即生效 [p.40]
- **使能 en**: else if (en) q <= d; 一仅 en=1 时更新 [p.41]
- **锁存器**: always_latch, 电平敏感, 尽量避免! [p.42]
- **case 组合**: 必须 default 覆盖所有情况, 否则产生锁存器 [p.45-46]
- **casez**: ? 为无关位, 实现优先级编码 [p.47]

阻塞 vs 非阻塞 (必考!)

- **非阻塞 <=**: 并行同时执行 [p.48-49]
- **阻塞 =**: 顺序依次执行 [p.48-49]
- **时序规则**: always_ff + <= / always_comb + = / assign + = [p.49]

FSM 与模块化

- **FSM 三模块**: (1) 次态逻辑 (comb) (2) 状态寄存器 (ff) (3) 输出逻辑 (comb) [p.50]
- **FSM 状态定义**: typedef enum logic [1:0] {S0,S1,S2} statetype; [p.52]
- **FSM 模块 1**: always_ff ... state <= nextstate; 一时序, <= [p.52]

- **FSM 模块 2:** `always_comb case(state) ...` 一组合, = [p.52]
- **parameter:** `#(parameter width = 8)` 位宽可配置 [p.53]

Testbench

- **Testbench 类型:** 简单/自检/测试向量 (Simple / Self-checking / Testvectors) [p.54]
- **\$readmemb:** 从文件读二进制测试向量 [p.63]
- **\$display:** 打印调试/错误信息 [p.58]
- **上升沿/下降沿:** 上升沿施加输入, 下降沿比较输出 (等稳定) [p.60,64–65]
- **\$finish:** 结束仿真 [p.66]

12 实验 3 关联：单周期 CPU 中的 HDL 应用

- **模块层次化**：CPU 顶层由 ALU、寄存器文件、控制器、指令存储器、数据存储器等模块实例化构成
- **组合逻辑**：ALU 运算、多路选择器 (assign / always_comb)
- **时序逻辑**：PC 寄存器、寄存器文件 (always_ff + <=)
- **FSM 控制器**：主控状态机用三模块结构实现指令译码-执行流程
- **Testbench**：编写测试向量验证每条指令的执行结果
- **参数化**：数据位宽 (32-bit)、地址位宽等通过 parameter 配置